CS23633 CLOUD COMPUTING

Cloud-Based Vision Analyzer: Deployment of a Machine Learning Image Analysis System on Microsoft Azure

A PROJECT REPORT

Submitted by

2116230701374 – VELAN ARUL

2116230701513 – VEERANDIRA SARAN

MAY 2026

BONAFIDE CERTIFICATE

Certified that this project report “Cloud-Based Vision Analyzer: Deployment of a Machine Learning Image Analysis System on Microsoft Azure” is the bonafide work of “VELAN A & VEERANDIRA SARAN” who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

ABSTRACT

Vision Analyzer is a machine learning-based web application designed to analyze and interpret visual data from images. The aim of the project is to automate the process of understanding image content by detecting objects and generating simple descriptions. The system is developed using Python and Flask, providing an easy-to-use interface where users can upload images or capture input through a webcam. Once an image is provided, the application processes it and returns relevant information, making it easier to understand the content without manual analysis.

The core functionality of the project is powered by deep learning models. YOLOv5 is used for object detection, which identifies multiple objects in an image along with their labels and confidence scores. For image captioning, a transformer-based model is used to generate a brief textual description of the scene. These models are pretrained on large datasets such as COCO and caption datasets, allowing them to recognize patterns and generalize well to new images. The system is designed in a modular way, separating detection and captioning components and integrating them through the backend.

The project also addresses challenges related to deploying machine learning applications. Overall, Vision Analyzer highlights the practical use of computer vision and machine learning to build an intelligent image analysis application.

INTRODUCTION

In recent years, the rapid growth of digital data has led to an increasing amount of visual content in the form of images and videos. Understanding and analyzing this visual data manually is time-consuming and often inefficient. With the advancement of artificial intelligence, particularly in machine learning and deep learning, it has become possible to automate the process of interpreting visual information. Computer vision techniques enable systems to extract meaningful insights from images, making them highly useful in various real-world applications.

This project, titled Vision Analyzer, focuses on developing a web-based application that can automatically analyze images and provide useful information about their content. The system is designed to detect objects present in an image and generate a descriptive caption that summarizes the scene. By integrating object detection and image captioning techniques, the application aims to simulate a basic level of human-like understanding of visual data. The user can interact with the system through a simple interface by uploading an image or using a webcam as input.

The implementation of this project is based on machine learning models that have been trained on large datasets. YOLOv5 is used for efficient object detection, while a transformer-based model is used for generating captions. The application is built using Python and Flask, and is deployed on the cloud using Microsoft Azure. This project demonstrates how machine learning and web technologies can be combined to create an intelligent system capable of understanding and describing images, while also addressing practical challenges such as model size and deployment constraints.

LITERATURE REVIEW

Recent advancements in the field of computer vision and machine learning have significantly improved the ability of systems to understand and interpret visual data. Earlier image analysis techniques relied on traditional image processing methods, which required manual feature extraction and were limited in accuracy and scalability. With the introduction of deep learning, especially convolutional neural networks (CNNs), image recognition and object detection have become more efficient and reliable. Models such as Region-Based CNN (R-CNN), Fast R-CNN, and Faster R-CNN laid the foundation for modern object detection systems.

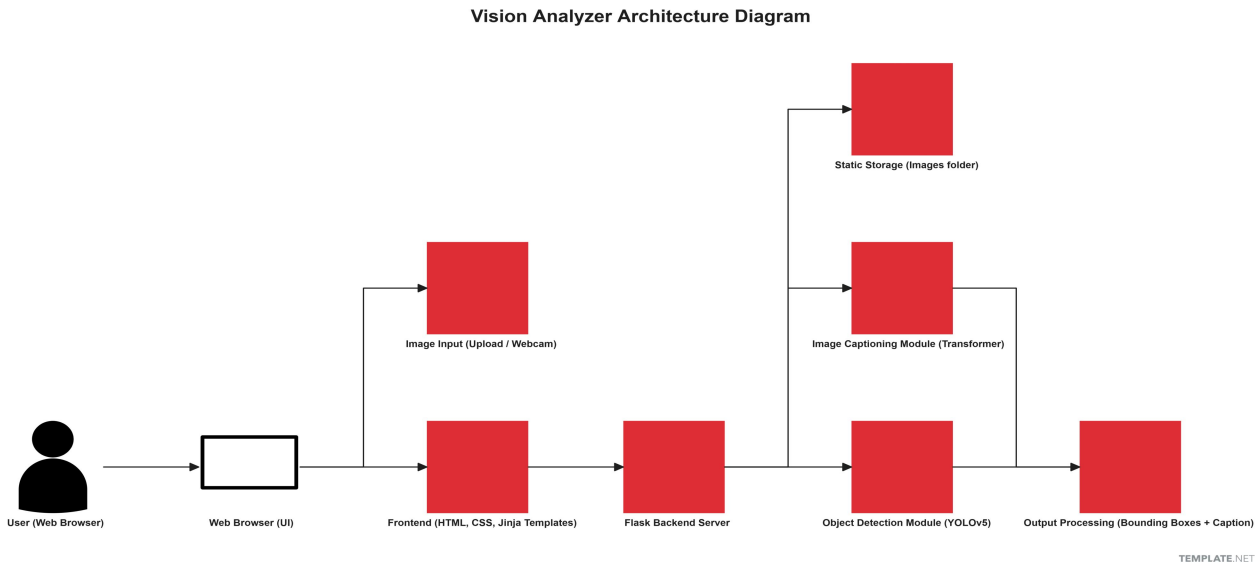
One of the most significant developments in object detection is the YOLO (You Only Look Once) algorithm, which performs detection in a single pass, making it much faster compared to earlier methods. YOLOv5, in particular, provides a good balance between speed and accuracy, making it suitable for real-time applications. In parallel, image captioning has evolved with the use of deep learning techniques that combine computer vision and natural language processing. Traditional methods used template-based approaches, but modern systems utilize encoder-decoder architectures and transformer-based models to generate more natural and meaningful descriptions of images.

Recent models such as BLIP (Bootstrapped Language Image Pretraining) have further improved the quality of image captioning by integrating vision and language understanding into a unified framework. These models are trained on large-scale datasets such as COCO and Conceptual Captions, enabling them to generalize well to new data. . This project builds upon these advancements by combining YOLOv5 for object detection and a transformer-based model for captioning within a web-based application.

PROPOSED SYSTEM

The proposed system is a machine learning-based web application that performs automatic image analysis by combining object detection and image captioning techniques. The system allows users to upload an image or use a webcam as input, which is then processed by the backend using pretrained deep learning models. YOLOv5 is used to detect and identify objects present in the image, while a captioning model generates a descriptive summary of the scene. The application is built using Python and Flask, with a modular structure separating detection and captioning components. The final output includes detected objects, confidence scores, and a generated caption displayed through a user-friendly web interface.

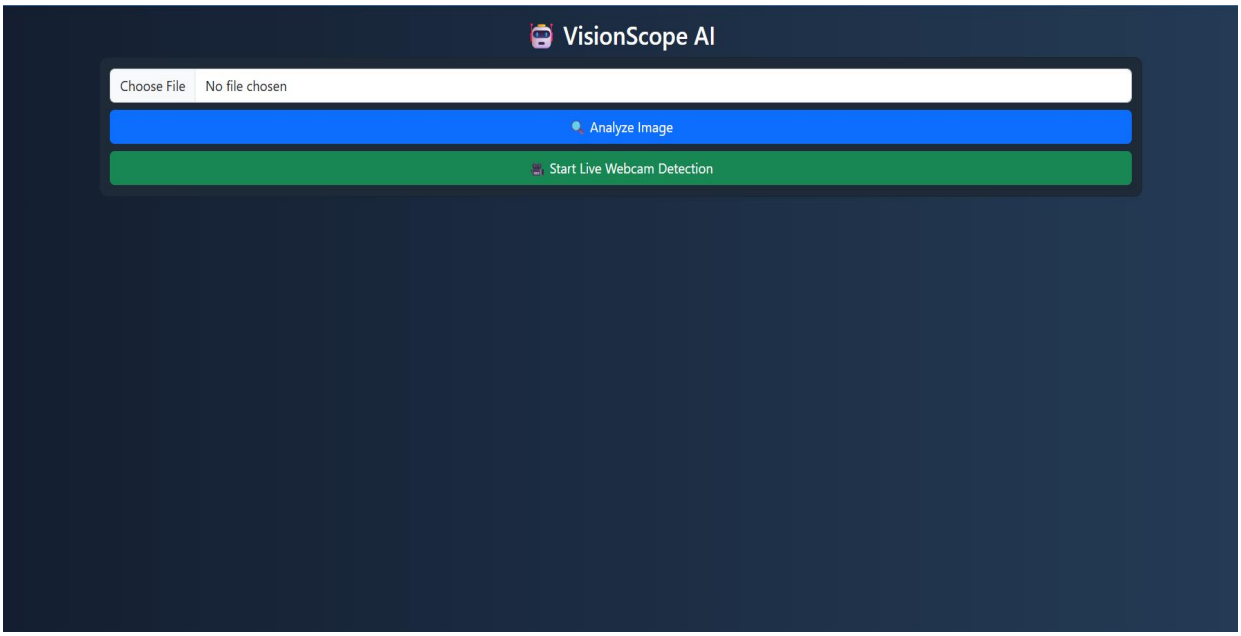
ARCHITECTURE DIAGRAM



MODULES DESCRIPTION

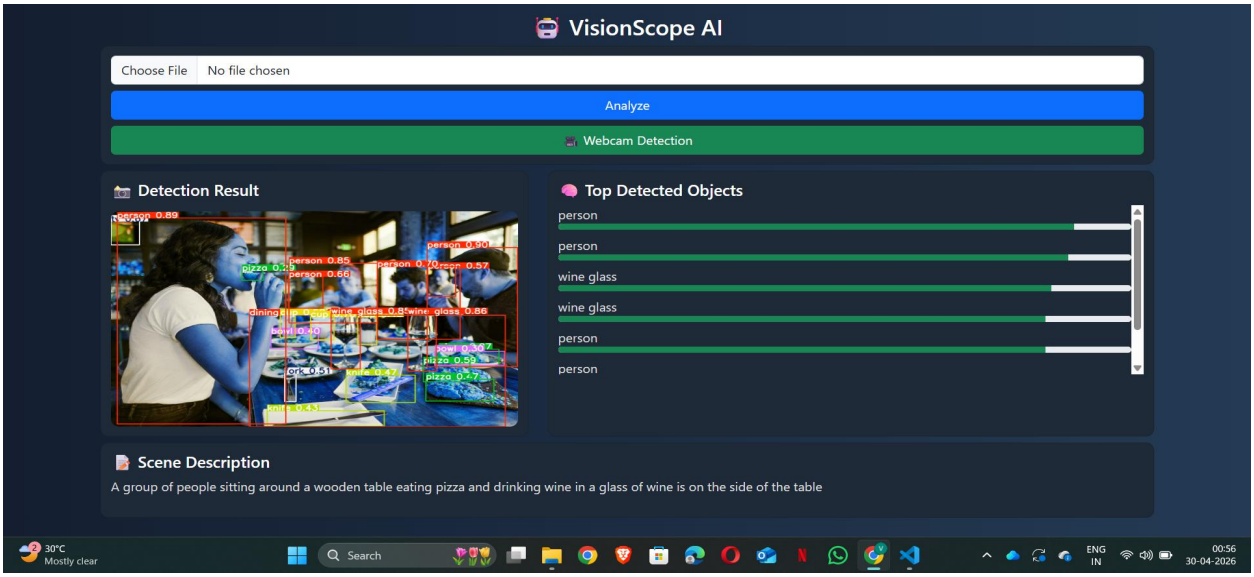
MODULE 1 : OBJECT DETECTION MODULE

The Object Detection Module is responsible for identifying and locating objects present in the input image. This module uses the YOLOv5 (You Only Look Once) deep learning model, which is known for its speed and accuracy in real-time detection tasks. When an image is provided by the user, it is passed to the model, which processes the image in a single pass and predicts the objects present along with their bounding boxes and confidence scores. The detected objects are then labeled and visually highlighted on the image. This module plays a key role in extracting structured information from the image by recognizing multiple objects simultaneously and providing precise localization.



MODULE 2 : Image Captioning Module

The Image Captioning Module is responsible for generating a meaningful textual description of the input image. This module uses a transformer-based deep learning model that combines computer vision and natural language processing techniques. The model analyzes the visual features of the image and generates a human-like sentence that describes the scene, including objects and their relationships. The caption provides a high-level understanding of the image content, complementing the object detection results. This module enhances the overall functionality of the system by converting visual data into natural language, making it easier for users to interpret the image.



IMPLEMENTATION AND RESULTS

EXPERIMENTAL SETUP

The experimental setup for the Vision Analyzer project focuses on implementing and testing the system in a local development environment and deploying it on a cloud platform. The application was developed using Python with the Flask framework as the backend, and HTML, CSS, and Jinja templates for the frontend interface. The system was executed on a standard personal computer with sufficient RAM and storage to handle the required libraries and pretrained models. The development environment included tools such as Visual Studio Code and a virtual environment for managing dependencies.

For the machine learning component, pretrained models were used to perform object detection and image captioning. The YOLOv5 model was utilized for detecting objects in images, providing bounding boxes, labels, and confidence scores. A transformer-based model was used for generating captions based on the input image. The models were integrated into the application through separate modules, ensuring a modular and maintainable design. Required libraries such as PyTorch, OpenCV, and Transformers were installed and configured to support the functioning of these models during local execution.

To evaluate the system, different test images were provided as input through the web interface, including images with multiple objects and varying complexity. The output was observed in terms of accuracy of detected objects, clarity of bounding boxes, and relevance of the generated captions. Additionally, the application was deployed on the cloud using Microsoft Azure App Service to test its functionality in a real-world environment. Due to resource limitations in the cloud, lightweight

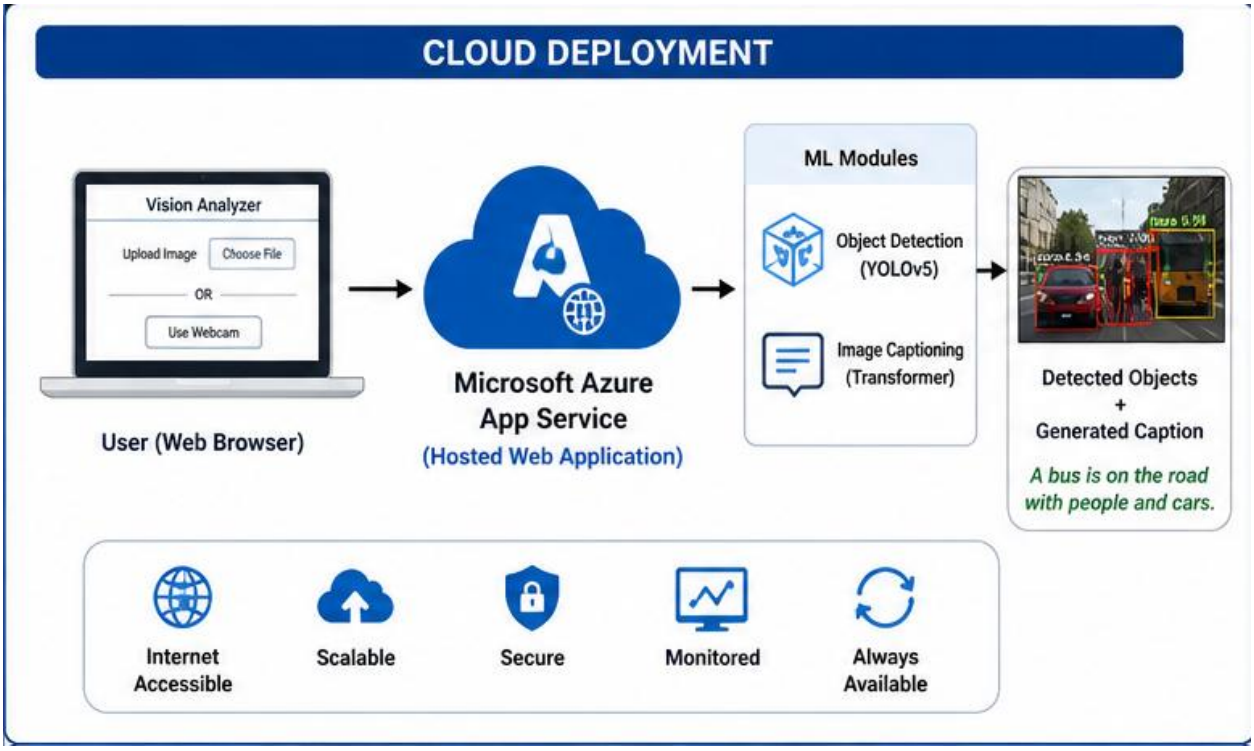
configurations were used to ensure successful deployment while maintaining core functionality.

CLOUD DEPLOYMENT

The Vision Analyzer application was deployed on the cloud using Microsoft Azure App Service to make it accessible over the internet. Cloud deployment enables users to interact with the application from any location without requiring local installation. The deployment process involved packaging the application files, including the Flask backend, frontend templates, and required dependencies, into a compressed archive and uploading it to the Azure Web App service.

An App Service plan was created to define the computational resources required for running the application. The web application was configured to use a Python runtime environment, and necessary settings such as startup commands and environment variables were defined. The deployment was carried out using Azure CLI, which allowed seamless uploading and building of the application on the cloud platform. Once deployed, the application was accessible through a public URL, enabling real-time interaction with the system.

Due to the limitations of the free-tier cloud environment, certain optimizations were implemented. Heavy machine learning models were replaced with lightweight placeholders to ensure smooth deployment and execution. Despite these constraints, the deployed system successfully demonstrates the integration of machine learning with cloud computing, highlighting scalability, accessibility, and ease of deployment.



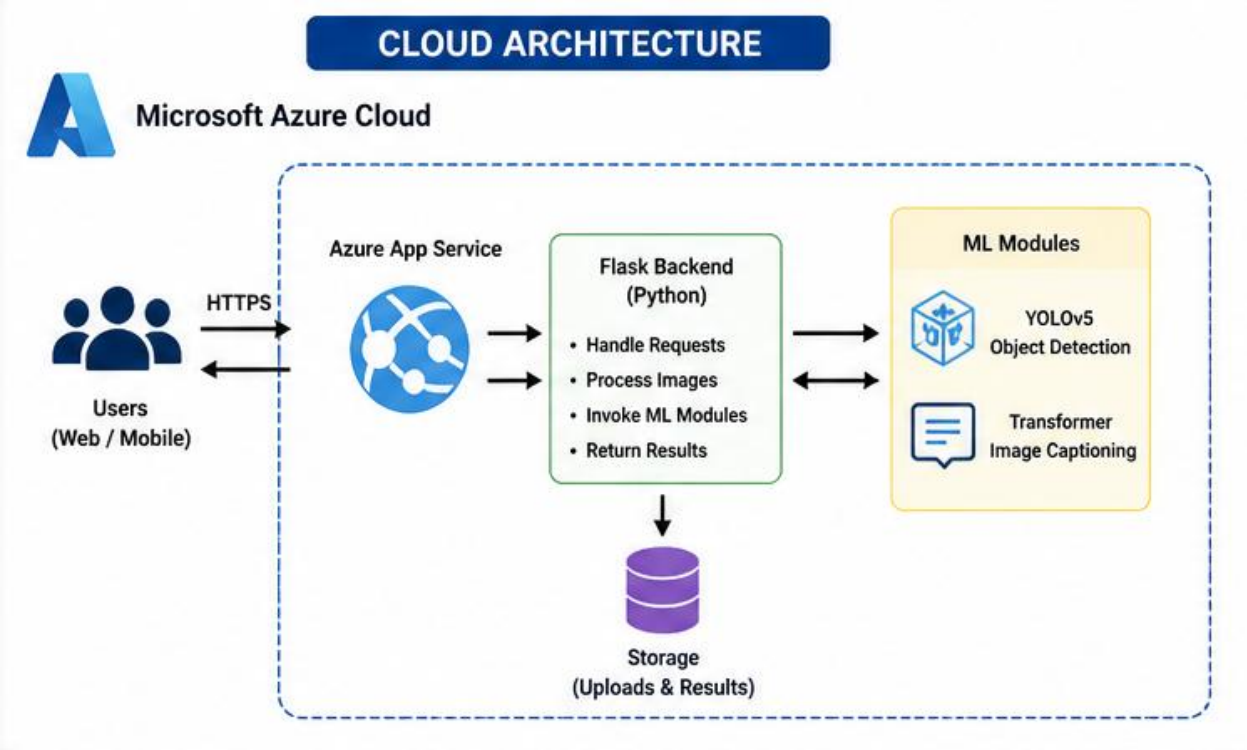
CLOUD ARCHITECTURE

The cloud architecture of the Vision Analyzer system is designed to ensure efficient handling of user requests and seamless integration of machine learning components. The architecture consists of a client-server model, where the user interacts with the application through a web browser. The frontend interface sends requests to the Flask backend hosted on Azure App Service.

The backend processes incoming requests, handles image uploads, and invokes the machine learning modules for object detection and caption generation. The processed results, including annotated images and generated captions, are then sent

back to the frontend for display. Static files such as uploaded images and output results are managed within the application's storage structure.

Azure App Service acts as the central component, providing hosting, scalability, and runtime support for the application. This architecture ensures that the system is modular, scalable, and capable of handling multiple user requests efficiently. It also separates concerns between user interaction, application logic, and machine learning processing.



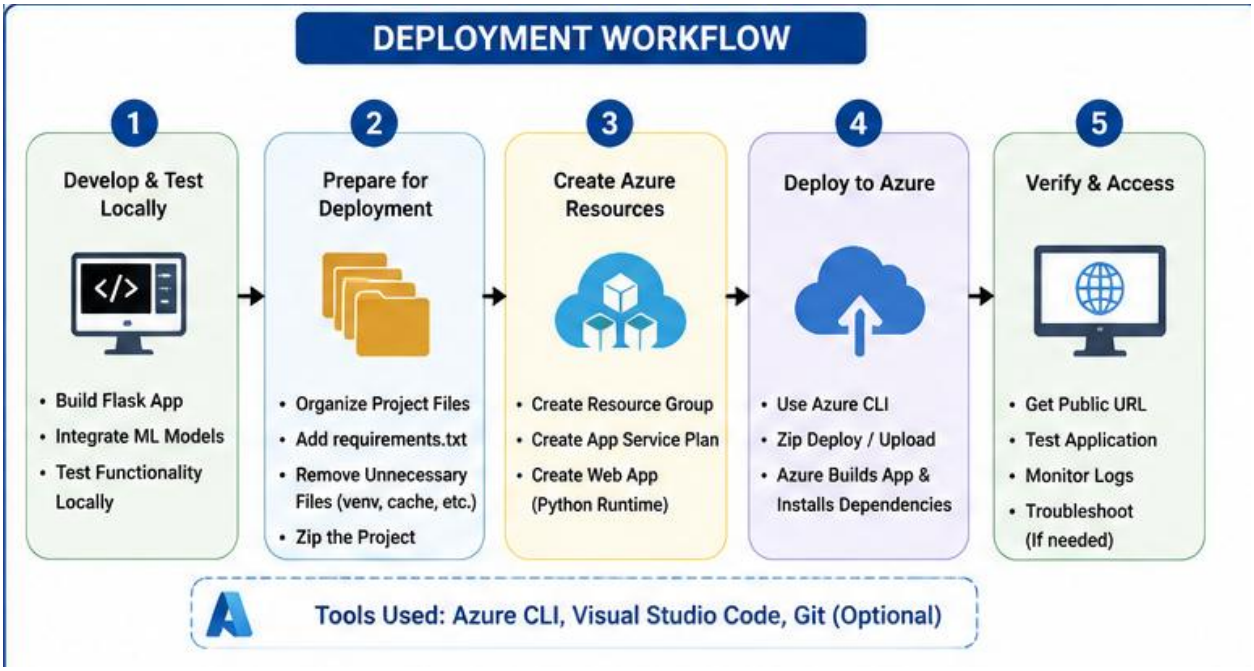
DEPLOYMENT WORKFLOW

The deployment workflow for the Vision Analyzer project involves several steps, starting from local development to cloud hosting. Initially, the application was developed and tested in a local environment using a virtual environment to manage

dependencies. After ensuring proper functionality, unnecessary files such as virtual environments and cache folders were removed to optimize the project size.

The cleaned project was then compressed into a ZIP file and deployed to Azure using command-line tools. Azure CLI commands were used to create a resource group, define an App Service plan, and deploy the web application. During deployment, Azure automatically installed the required dependencies specified in the requirements file and configured the runtime environment.

After successful deployment, the application was tested using the provided URL to verify its functionality. Any errors encountered during deployment were resolved by adjusting dependencies and simplifying configurations. This workflow demonstrates a practical approach to deploying machine learning applications on cloud platforms, ensuring reliability and accessibility.

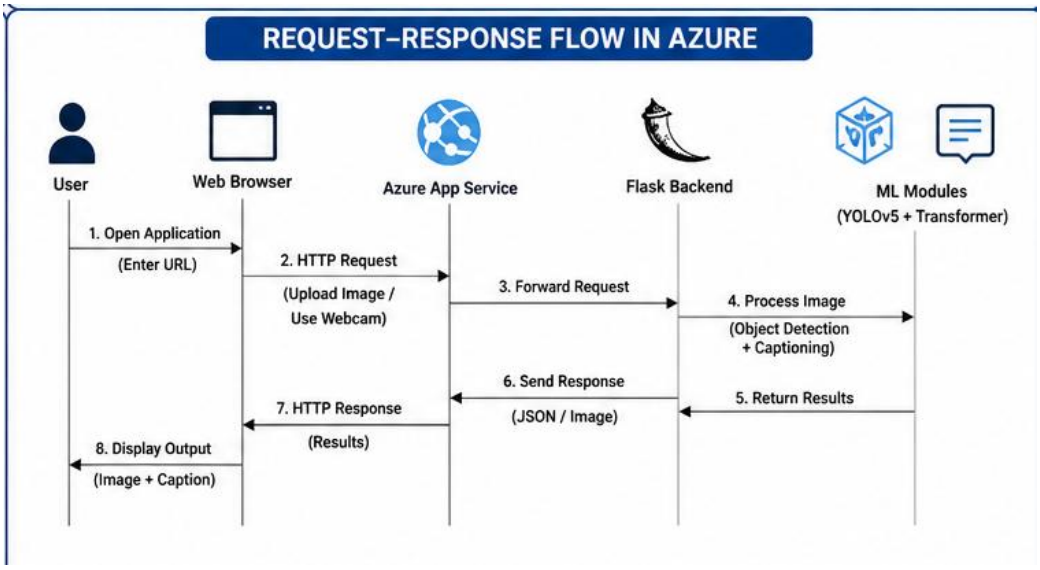


REQUEST RESPONSE FLOW IN AZURE

The request–response flow of the Vision Analyzer application on the Azure platform follows a client-server architecture. When a user accesses the application through a web browser, a request is sent to the hosted web application on Azure App Service. The frontend interface captures user input, such as an uploaded image, and sends it to the backend server for processing.

The Flask backend receives the request and processes the image using the integrated machine learning modules. Object detection and caption generation are performed, and the results are prepared for display. The processed output, including annotated images and captions, is then sent back to the frontend as a response.

Azure efficiently handles this communication by managing the server environment, ensuring that requests are processed reliably and responses are delivered quickly. This flow highlights how cloud platforms enable seamless interaction between users and machine learning applications deployed on remote servers.



RESULTS

The results of the Vision Analyzer project demonstrate the system's ability to effectively analyze visual data through both image and video inputs. For image-based analysis, the system successfully detects multiple objects present in the uploaded image using the YOLOv5 model. The output includes clearly marked bounding boxes around detected objects along with their corresponding labels and confidence scores. In addition to object detection, the system generates a descriptive caption that summarizes the overall scene. The results show that the model performs well on images containing common objects, providing accurate and meaningful outputs that help in understanding the image content.

For video-based analysis using the webcam, the system processes frames in real time and performs continuous object detection. As the webcam captures live video, each frame is analyzed, and detected objects are highlighted dynamically with bounding boxes and labels. This demonstrates the real-time capability of the YOLOv5 model, making the system responsive and interactive. Although image captioning is primarily applied to static images, the video output effectively showcases the system's ability to handle continuous input and perform detection with minimal delay.

Overall, the system produces reliable results for both static and dynamic inputs. The combination of object detection and caption generation enhances the interpretability of visual data. While the local implementation provides full functionality with accurate detection and captioning, the cloud deployment ensures accessibility, though with certain limitations due to resource constraints. The results validate the effectiveness of integrating machine learning models into a web-based application for practical image and video analysis.

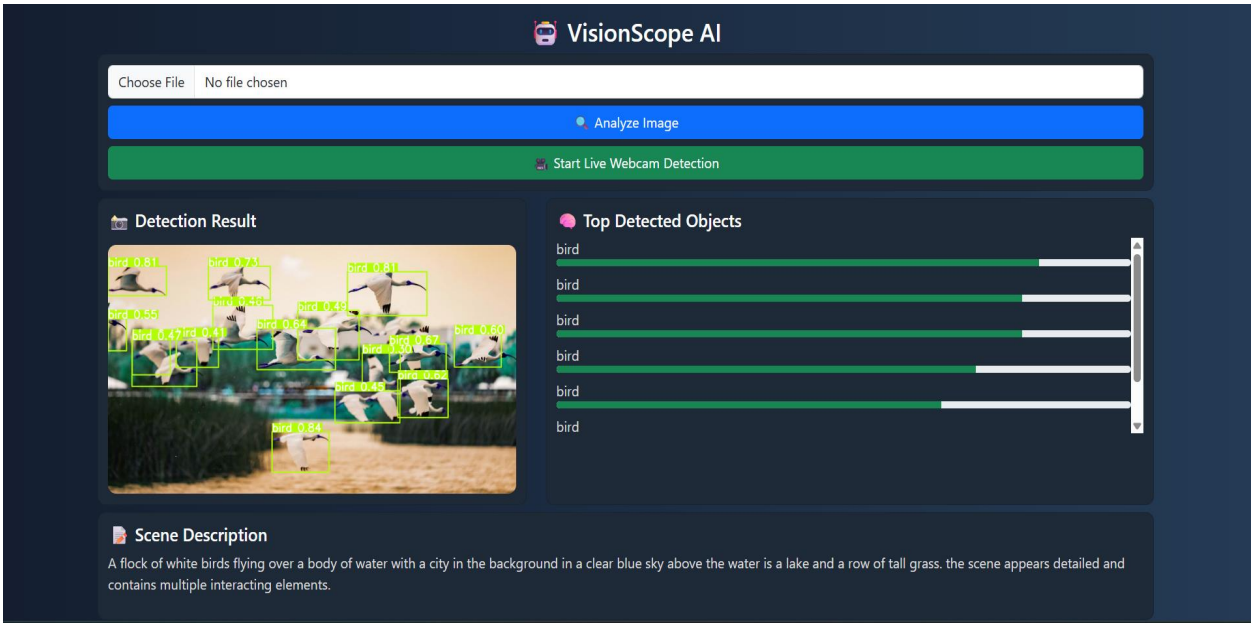
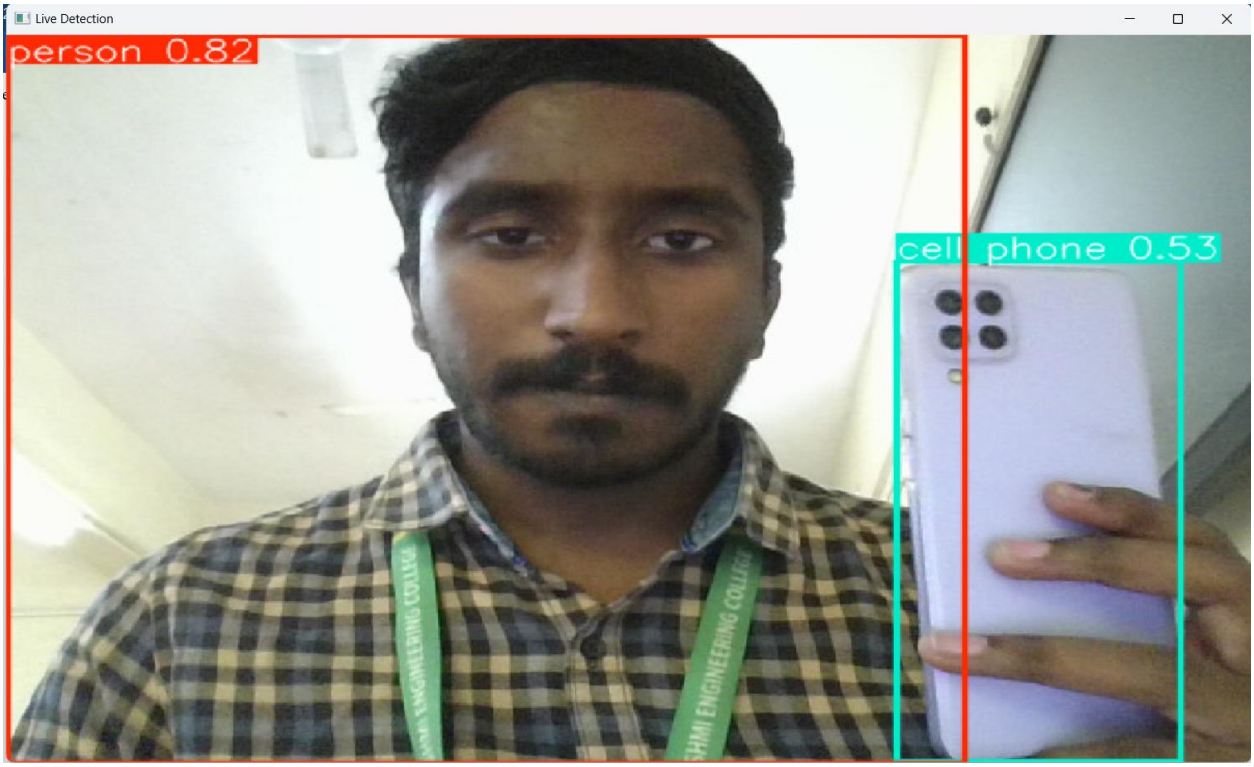


IMAGE ANALYSIS AND REVIEW GENERATION



VIDEO ANALYSIS AND OBJECT DETECTION

CONCLUSION AND FUTURE WORK

The Vision Analyzer project successfully demonstrates the integration of machine learning and web technologies to build an intelligent system capable of understanding visual data. The application was designed to perform object detection and image captioning, allowing users to gain meaningful insights from images and live video input. By utilizing pretrained deep learning models such as YOLOv5 for object detection and a transformer-based model for caption generation, the system is able to identify multiple objects, provide confidence scores, and generate descriptive summaries of scenes. The use of Flask for backend development and a simple frontend interface ensures that the system is both functional and user-friendly.

Throughout the development of this project, several practical challenges were encountered, including handling large model sizes, managing dependencies, and deploying the application on a cloud platform. These challenges were addressed by adopting a modular design approach and implementing lightweight configurations for cloud deployment using Microsoft Azure. While the local implementation supports full functionality with accurate detection and captioning, the deployed version demonstrates how such systems can be adapted for real-world use despite resource limitations. This project highlights the importance of combining technical knowledge with practical problem-solving to build scalable and efficient machine learning applications.

For future work, the system can be enhanced in several ways to improve performance and usability. One major improvement would be enabling full-scale deployment of deep learning models on the cloud by integrating external APIs or using more powerful cloud resources. The accuracy of object detection and caption

generation can be further improved by fine-tuning the models on custom datasets. Additionally, support for real-time video captioning and advanced features such as object tracking and scene understanding can be added. The user interface can also be enhanced to provide a more interactive and visually appealing experience. Overall, Vision Analyzer has strong potential for further development into a more robust and scalable intelligent vision system.

REFERENCES

[1] Brownlee, J. (2016) ‘Deep Learning for Computer Vision’, Machine Learning Mastery, Vol.1, No.1, pp.1-20.

[2] Chen, T., Kornblith, S., Norouzi, M. and Hinton, G. (2020) ‘A Simple Framework for Contrastive Learning of Visual Representations’, Proceedings of the International Conference on Machine Learning, Vol.119, pp.1597-1607.

[3] Devlin, J., Chang, M.W., Lee, K. and Toutanova, K. (2019) ‘BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding’, Proceedings of NAACL-HLT, Vol.1, pp.4171-4186.

[4] Dosovitskiy, A., Beyer, L., Kolesnikov, A. et al. (2021) ‘An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale’, International Conference on Learning Representations, Vol.1, pp.1-22.

[5] Everingham, M., Van Gool, L., Williams, C.K.I., Winn, J. and Zisserman, A. (2010) ‘The Pascal Visual Object Classes (VOC) Challenge’, International Journal of Computer Vision, Vol.88, No.2, pp.303-338.

[6] He, K., Zhang, X., Ren, S. and Sun, J. (2016) ‘Deep Residual Learning for Image Recognition’, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Vol.1, pp.770-778.

[7] Howard, A.G., Zhu, M., Chen, B. et al. (2017) ‘MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications’, arXiv preprint arXiv:1704.04861, Vol.1, pp.1-9.

[8] Jocher, G., Stoken, A., Borovec, J. et al. (2021) ‘YOLOv5 by Ultralytics’, GitHub Repository, Vol.1, pp.1-10.

[9] Lin, T.Y., Maire, M., Belongie, S. et al. (2014) ‘Microsoft COCO: Common Objects in Context’, European Conference on Computer Vision, Vol.8693, pp.740-755.

[10] Liu, W., Anguelov, D., Erhan, D. et al. (2016) ‘SSD: Single Shot MultiBox Detector’, European Conference on Computer Vision, Vol.9905, pp.21-37.

[11] Radford, A., Kim, J.W., Hallacy, C. et al. (2021) ‘Learning Transferable Visual Models From Natural Language Supervision’, International Conference on Machine Learning, Vol.139, pp.8748-8763.

[12] Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2016) ‘You Only Look Once: Unified, Real-Time Object Detection’, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Vol.1, pp.779-788.

[13] Szegedy, C., Liu, W., Jia, Y. et al. (2015) ‘Going Deeper with Convolutions’, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Vol.1, pp.1-9.

[14] Vaswani, A., Shazeer, N., Parmar, N. et al. (2017) ‘Attention is All You Need’, Advances in Neural Information Processing Systems, Vol.30, pp.5998-6008.

[15] Wang, Z., Bovik, A.C., Sheikh, H.R. and Simoncelli, E.P. (2004) ‘Image Quality Assessment: From Error Visibility to Structural Similarity’, IEEE Transactions on Image Processing, Vol.13, No.4, pp.600-612.